

- ▶ **P**, which contains those problems which can be solved in polynomial time, so the **tractable** ones.
- ▶ **EXP**, which contains the whole of **P**, but also some problems which *cannot* be solved in polynomial time, intrinsically requiring exponential time (and as such, **intractable**).

It's now time to study the “border” between tractability and intractability:

Between **P** and **EXP**, one can define *many other* classes. i.e. there are many ways of defining a class **A** such that

$$\mathbf{P} \subseteq \mathbf{A} \subseteq \mathbf{EXP}$$

This is a formidable tool to classify those problems in **EXP** for which *we do not know* whether they are in **P** (and there are *so many* of them).

Creating vs. Verifying

- ▶ Very often, the language we would like to classify can be written as follows:

$$\mathcal{L} = \{x \in \{0, 1\}^* \mid \exists y \in \{0, 1\}^{p(|x|)}. (x, y) \in \mathcal{A}\}$$

where $p : \mathbb{N} \rightarrow \mathbb{N}$ and $\mathcal{A}$ is a set of pairs of strings.

This mathematical expression describes a fundamental way to define languages in the context of complexity theory, specifically defining the class **NP** (Nondeterministic Polynomial time) through the lens of **verification**. Here is a breakdown of what each component of that notation means:

1. The Core Definition: $\mathbf{L} = \{x \in \{0, 1\}^* \mid \dots\}$

This states that a language L consists of all strings x that satisfy a specific condition (These strings can be seen as Strings that encodes the specific problem you are trying to solve). In this context, the goal is to determine if x belongs to the "tractable" or "intractable" side of computational complexity.

2. The Certificate: $\exists y \in \{0, 1\}^{p(|x|)}$

This is the most important part of the definition. It says that for x to be in the language L , there must **exist** a string y , called a **certificate** (or witness). y can be seen as a solution to the problem encoded by x

- **Polynomial Length:** The length of this certificate y is restricted by $p(|x|)$, which is a **polynomial function** of the length of the input x . This means the "proof" or "solution" cannot be infinitely long; it must be relatively short compared to the problem size.

3. The Test: $(x, y) \in A$

This represents the **verification process**. A is a set of pairs (x, y) that "pass the test".

A can be seen as an algorithm that verifies if y is a valid solution for the problem encoded in x

- If we are given the input x and the correct certificate y , we can check if (x, y) belongs to A .
- In the definition of NP, the "test" A must be **decidable in polynomial time**. This means that while finding the certificate y might be very difficult, **verifying** that y is a valid solution for x is easy (tractable).

Summary "Take-away" Message

The notation formalizes the difference between **creating** a solution and **verifying** one.

- **To check if $x \in L$:** You would potentially have to check every possible y . Since y can be any string up to a certain length, there are **exponentially many** possible certificates to check, making this direction potentially hard.
- **To verify $x \in L$:** If someone gives you the correct y , you can run the "test" A very quickly to see if it's correct.

Real-World Examples

- **Maximum Independent Set:** x is a graph and a number k . The certificate y is a specific set of vertices. It is easy to verify if that set is independent and has size k , but hard to find such a set among all possibilities.
- **Composite Numbers:** x is a number n . The certificate y is a pair of factors (m, l) . It is easy to multiply $m \cdot l$ to see if they equal n , but finding those factors for a very large number is traditionally difficult.
- **Traveling Salesman:** x is a map of cities and a distance k . The certificate y is the specific order of cities visited. It is easy to sum the distances of that specific route, but hard to find the shortest route among all possible permutations.

► With the hypotheses above:

- M is said to be the **verifier** for $\mathcal{L}$.
- Any $y \in \{0, 1\}^{p(|x|)}$ such that $\mathcal{M}(\perp(x, y)\perp) = 1$ is said to be a **certificate** for x .

Essentially, the class **NP** is a subset of **EXP** consisting of problems for **which** it is possible to **verify** a solution in **polynomial** time.

In this context, the "solution" is technically referred to as a **certificate** (or witness) of polynomial length, and the process of checking it is performed by a **polynomial-time verifier**. While finding such a certificate might take exponential time, the defining characteristic of NP is that the verification itself is efficient.

- Differently from **P** and **EXP**, the class **NP** does not have a natural counterpart as a class of *functions*.

Theorem

$$\mathbf{P} \subseteq \mathbf{NP} \subseteq \mathbf{EXP}$$

- **Any Problem in P**
 - Every problem in **P** can be seen as a problem in **NP**.

Examples Problems in NP and in P

here are the six examples of problems in the **NP** complexity class, including their descriptions and the certificates used to verify them:

1. **Maximum Independent Set:** In its decision form, this problem asks if a given undirected graph $G = (V, E)$ and a natural number k admit an **independent set** $W \subseteq V$ with a cardinality of at least k . The **certificate** is the set W itself, as checking if W is independent can be done easily in polynomial time.
2. **Subset Sum:** This problem asks whether, given a sequence of natural numbers $n_1, \dots, n_m$ and a target number k , there exists a subset of those numbers that sums exactly to k . The **certificate** is the subset I of indices; verifying the sum requires only simple additions and comparisons.
3. **Composite Numbers:** This problem involves determining whether a given natural number n is composite (meaning it is not a prime number). The **certificate** is the **factorization** of n , specifically a pair of natural numbers (m, l) both greater than 2 such that $n = m \cdot l$. Note that while this is in **NP**, it was recently proven to also be in **P** via the AKS algorithm.
4. **Traveling Salesman Problem:** Given a list of cities, the distances between them, and a natural number k , this problem asks if there is a route that visits every city exactly once and returns to the start within a total distance of at most k . The **certificate** is the specific **order** in which the cities are visited, which can be checked in polynomial time.
5. **Decisional Linear Programming:** This problem asks whether a sequence of m linear inequalities $\sum_{i=1}^n a_i x_i \leq b$, where the coefficients $a_1, \dots, a_n, b$ are in $\mathbb{Q}$, with rational coefficients over n variables has a **rational assignment** that makes all inequalities true. The **certificate** is the rational assignment itself, which is easily checked for correctness. Like composite numbers, this problem is also known to be in **P** (ellipsoid algorithm).
6. **Decisional 0/1 Linear Programming:** Similar to the previous problem, this asks if there is an assignment of only **zeros and ones** to the variables that satisfies all the linear inequalities. The **certificate** is the specific 0/1 assignment. Unlike the general rational version, this problem is currently not known to be in **P**.

Nondeterministic Turing Machines

- The class **NP** can also be defined using a variant of Turing machines, called the *nondeterministic* Turing machines (NDTM for short).

- ▶ The only differences between a NDTM and an ordinary TM is that the former has:
 - ▶ Two transition functions δ_0 and δ_1 rather than just one. *At every step*, the machine chooses nondeterministically one between the two transition functions and proceed according to it
 - ▶ A special state q_{accept} .
- ▶ We say that a NDTM M :
 - ▶ **Accepts** the input $x \in \{0, 1\}^*$ iff *there exists* one among the many possible evolutions of the machine M when fed with x which makes it reaching q_{accept} .
 - ▶ **Rejects** the input $x \in \{0, 1\}^*$ iff *none* of the aforementioned evolutions leads to q_{accept} .

Acceptance and Rejection ($M(x) = 1$)

The behavior of an NDTM is defined by its possible "evolutions" or paths:

- **Acceptance** ($M(x) = 1$): The machine is said to accept an input string x if **at least one** of its many possible evolutions reaches a special state called q_{accept} .
- **Rejection**: The machine rejects the input if **none** of the possible evolutions reach q_{accept} .

Working in Time T

For an NDTM to be considered "time-bounded" by a function T , it must meet a very strict requirement: **every possible evolution** (regardless of the choices made) must reach either a halting state or q_{accept} within $c \cdot T(|x|)$ steps, where c is a constant and $|x|$ is the length of the input.

Defining the Class $\text{NDTIME}(T(n))$

The definition states that a language L belongs to the class $\text{NDTIME}(T(n))$ if there is an NDTM M that:

- Operates within the time bound T .
- Correctly identifies the language: it outputs 1 (accepts) if and only if the input x is actually a member of L .

Connection to NP

This definition is fundamental to understanding the class **NP** (Nondeterministic Polynomial time). The class **NP is the union of all NDTIME classes** where the time function T is a polynomial (n^c). Essentially, NP consists of all problems that can be solved by an NDTM in polynomial time.

Theorem

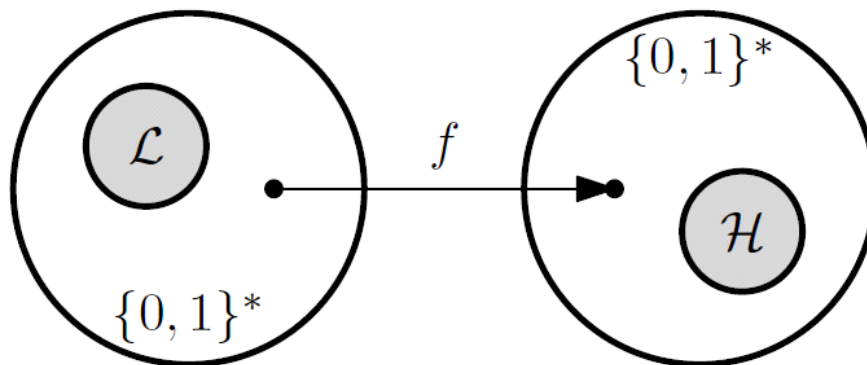
$$\text{NP} = \bigcup_{c \in \mathbb{N}} \text{NDTIME}(n^c)$$

Are all Problems in **NP** Equivalent?

- ▶ The Maximum Independent Set, a problem we already know about, is in **NP**.
- ▶ The language of, say, palindrome words, is easy to be proved in **P**, thus in **NP**.
- ▶ Intuitively, however, the inherent difficulties of solving the two problems *should be* different.
- ▶ What can we thus conclude from the fact that a language $\mathcal{L}$ is **in** the class **NP**?
 - ▶ Not much, actually! We can only conclude that it is not *too* complicated to solve it.
- ▶ We need something else, namely a (pre-order) relation between languages such that two languages being in relation tells us something precise about the **relative** difficulty of deciding them.

Reductions

;))



The **polynomial-time reduction** is a fundamental tool in computational complexity used to compare the relative difficulty of different problems.

Here is a breakdown of what this definition means and why it is important:

1. The Core Concept: "Translation"

At its heart, a reduction is a way of **translating** one problem into another.

- **The Function (f):** This is the "translator." It takes an input x (an instance of problem L) and transforms it into an input $f(x)$ (an instance of problem H).
- **Polynomial-Time:** For the reduction to be useful, this translation must be efficient.

Specifically, the function f must be computable in **polynomial time**, meaning the translation doesn't take an "unreasonable" amount of time relative to the size of the input.

2. The "If and Only If" Condition

The most critical part of the definition is the condition: $x \in L \text{ iff } f(x) \in H$. This means that the "yes/no" answer to the original problem (L) must be preserved by the translation into the second problem (H):

- If the answer to x in problem L is "Yes," then the answer to $f(x)$ in problem H must also be "Yes".
- If the answer to x in problem L is "No," then the answer to $f(x)$ in problem H must also be "No".

3. Understanding the Notation $L \leq_p H$

The notation $L \leq_p H$ is read as " L is **polynomial-time reducible to H** ".

- The " $\leq$ " symbol is used because this reduction tells us that **H is at least as difficult as L** .
- If you have a fast way to solve H , you can now solve L quickly by first translating L into H and then solving H .

4. Why This Matters

- **Classifying Difficulty:** Reductions allow scientists to group problems together. For example, if $L \leq_p H$ and H can be solved in polynomial time (it's in class **P**), then L must also be in **P**.
- **Defining NP-Completeness:** This concept is used to find the "hardest" problems in a class. A language H is called **NP-hard** if *every* problem in the class **NP** can be reduced to it ($L \leq_p H$ for all $L \in \text{NP}$).
- **Equivalence:** If two problems can be reduced to each other ($L \leq_p H$ and $H \leq_p L$), they are considered computationally **equivalent**. This is how "The Zoo of NP-complete Problems" is formed; despite looking different (like graph problems vs. logic problems), they are all essentially the same difficulty.

Reductions and Complexity

- ▶ If $\mathcal{L} \leq_p \mathcal{H}$, then $\mathcal{H}$ is at least as difficult as $\mathcal{L}$, at least as far as classes like **P** (or above it) are concerned.
 - ▶ If, e.g., $\mathcal{L} \leq_p \mathcal{H}$ and $\mathcal{H} \in \text{P}$, then also $\mathcal{L} \in \text{P}$: a way to decide if $x \in \mathcal{L}$ consists in translating it into $f(x)$ (which can be done in polynomial time), then checking whether $f(x) \in \mathcal{H}$.
- ▶ A language $\mathcal{H} \subseteq \{0, 1\}^*$ is said to be:
 - ▶ **NP-hard** if $\mathcal{L} \leq_p \mathcal{H}$ for every $\mathcal{L} \in \text{NP}$.
 - ▶ **NP-complete** if $\mathcal{H}$ is **NP-hard**, and $\mathcal{H} \in \text{NP}$.

NP-Hard Languages

A language H is **NP-hard** if every problem L in the class **NP** can be reduced to H in polynomial time ($L \leq_p H$).

- **Relative Difficulty:** This means that H is **at least as difficult** as any problem in the NP class.
- **Outside NP:** An NP-hard problem does not necessarily have to be in NP itself. It could be even more difficult, such as problems requiring exponential time (EXP).
- **The Power of Reduction:** If you found a polynomial-time algorithm for an NP-hard problem, you would effectively have an efficient way to solve *every* problem in NP by reducing them to H first.

NP-Complete Languages

A language H is **NP-complete** if it meets two specific conditions: it is **NP-hard**, and it is also a member of the **NP** class ($H \in NP$).

- **The Hardest in NP:** NP-complete problems are considered the "hardest" problems within the NP class because they are at least as hard as every other problem in that class.
- **Equivalence:** All NP-complete problems are computationally **equivalent** to one another. Because they are all NP-hard and all in NP, they can all be reduced to each other in polynomial time.
- **P vs. NP:** If **any** NP-complete problem is ever proven to be in **P** (solvable in polynomial time), then it would prove that **P = NP**. Currently, most scientists believe that P does not equal NP, meaning NP-complete problems are likely intractable.

To clarify this, it is helpful to look at the relationship between these complexity classes and what the " $\leq_p$ " (reduction) notation actually signifies.

1. P is a subset of NP

The first important point is that **the class P is a subset of the class NP ($P \subseteq NP$)**. This means that every problem that can be solved in polynomial time (P) is also, by definition, a problem in NP. A problem is in NP if a potential solution (a certificate) can be **verified** in polynomial time. If you can already **solve** a problem in polynomial time (which is what P means), you can certainly verify a solution in polynomial time just by re-solving it or ignoring the certificate.

2. The Meaning of "Difficulty" ($L \leq_p H$)

The reduction $L \leq_p H$ does **not** mean that L and H have the *same* difficulty. Instead, it means that **H is at least as difficult as L** .

- Think of it like this: if you have a "magic box" that solves the hard problem H instantly, you could use that box to solve the easier problem L after a quick (polynomial-time) translation.
- It is always possible to reduce a "simple" problem to a "harder" one. For example, you can reduce the task of "sorting a list" (which is in P) to a very complex problem like "the Traveling Salesman Problem" (which is NP-complete).

3. Are NP problems unsolvable in polynomial time?

- We know that all problems in **P** are solvable in polynomial time.
- We **do not know** if the problems in NP that are *outside* of P (like NP-complete problems) can be solved in polynomial time.
- Most computer scientists *believe* that $P \neq NP$, meaning they suspect there are problems in NP that truly cannot be solved in polynomial time, but this has never been proven.

4. Why L can be reduced to H

If H is **NP-hard**, then by definition **every** language L in NP must be reducible to it ($L \leq_p H$).

Since any language L that is in P is also in NP , it follows mathematically that L must be reducible to H .

In summary, the reduction doesn't claim L is as hard as H ; it claims that **if you can solve H , you can definitely solve L** . Since H is a "harder" category, it is perfectly capable of "handling" the simpler tasks found in L .

5. Does this mean every NP problem can be converted to any other NP problem?

No, not necessarily. The statement you provided defines H as being **NP-hard**. While it is true that every problem in NP can be "converted" (reduced) to an NP-hard problem, the reverse is not always true.

- **One-Way Street:** You can always reduce a "simple" NP problem (like a problem in P) to a "hard" NP-complete problem.
- **The "Hardest" Problems:** It is only when two problems are both **NP-complete** that they are considered computationally **equivalent**. In that specific case, they can both be reduced to each other, meaning they are essentially the same problem in different disguises.

6. Does solving one NP problem solve them all?

Yes, if that one problem is NP-hard. This is one of the most powerful results in complexity theory.

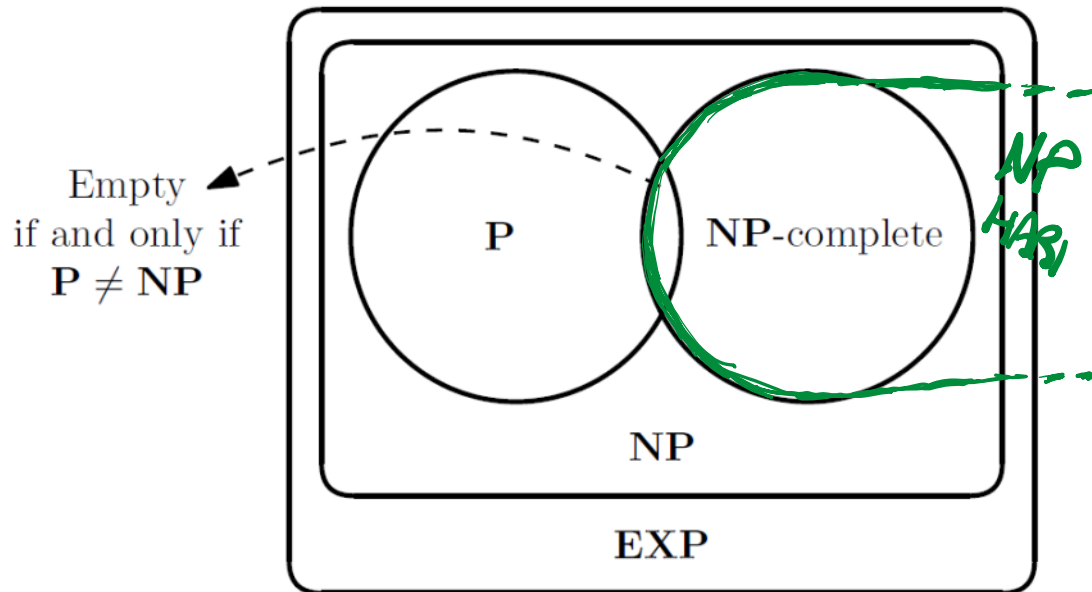
- If you find a polynomial-time algorithm for **any** NP-complete language H , then every single problem L in the class NP would also be solvable in polynomial time.
- This is because you could take any instance of an NP problem, use the reduction function (f) to turn it into an instance of H , and then solve it using your new fast algorithm.
- Mathematically, if an NP-hard problem is in P , then $P = NP$.

7. How is this possible when the problems are so different?

It seems counterintuitive that a problem about **logic** (like SAT) could be "the same" as a problem about **graphs** (like Independent Set) or **numbers** (like Subset Sum). However, they are linked by the underlying way we define "problems" in computer science:

- **The Universal Language of Turing Machines:** Every problem in NP is defined by a **verifier**—a program that runs on a Turing Machine and checks if a "certificate" (a solution) is valid in polynomial time.
- **The Cook-Levin Theorem:** This theorem proved that because Turing Machines follow very simple, local rules, the entire process of "checking a solution" can be translated into a giant **boolean logic formula** (the SAT problem).
- **The Chain of Reductions:** Once we know that every NP problem can be turned into a SAT formula, we can then find ways to turn those SAT formulas into graph structures or numerical equations.

In fact, there is a "Zoo" of thousands of NP-complete problems across almost every field of mathematics and science that are all linked this way; if you solve one, you solve them all.



Theorem

1. The relation $\leq_p$ is a pre-order (i.e. it is reflexive and transitive).
2. If $\mathcal{L}$ is NP-hard and $\mathcal{L} \in P$, then $P = NP$.
3. If $\mathcal{L}$ is NP-complete, then $\mathcal{L} \in P$ iff $P = NP$.

This theorem outlines the fundamental properties of polynomial-time reductions and their implications for the **P vs. NP** question. Below is a detailed explanation of each part of the theorem.

1. The relation $\leq_p$ is a pre-order

A **polynomial-time reduction** ($L \leq_p H$) means there is a function f , computable in polynomial time, such that for every string x , $x \in L$ if and only if $f(x) \in H$. To say this relation is a **pre-order** means it satisfies two specific mathematical properties:

- **Reflexivity** ($L \leq_p L$): Every language can be reduced to itself. This is proven by choosing the **identity function** ($f(x) = x$) as the reduction. Since the identity function simply returns the input, it is clearly computable in polynomial time and satisfies the condition that $x \in L \Leftrightarrow x \in L$.
- **Transitivity** (If $L \leq_p H$ and $H \leq_p J$, then $L \leq_p J$): If you can reduce problem L to H , and H to J , you can reduce L directly to J . This is proven using **function composition**. If f reduces L to H and g reduces H to J , then the composed function $g(f(x))$ reduces L to J . Because both f and g are computable in polynomial time, their composition is also polynomial time (applying one polynomial to the result of another still yields a polynomial).

2. If L is NP-hard and $L \in P$, then $P = NP$

This part of the theorem highlights the significance of **NP-hard** problems. A language L is defined as NP-hard if **every** language H in the class **NP** can be reduced to it in polynomial time ($H \leq_p L$ for all $H \in NP$).

The logic follows these steps:

- We already know that $P \subseteq NP$ (every problem solvable in polynomial time is also verifiable in polynomial time).
- If an NP-hard problem L is actually in P , it means we have a polynomial-time algorithm to solve it.
- Since every other problem H in NP reduces to L , we could solve any H by first translating it into an instance of L (which takes polynomial time) and then solving it using the algorithm for L .
- This would mean **every** problem in NP is also in P , leading to the conclusion that $NP \subseteq P$ and therefore $P = NP$.

3. If L is NP-complete, then $L \in P$ iff $P = NP$

An **NP-complete** problem is a "representative" of the hardest problems in NP . By definition, L is NP-complete if it is in NP and it is **NP-hard**. This part of the theorem is an "if and only if" (iff) statement, meaning it works in both directions:

- **Direction 1 ($L \in P \rightarrow P = NP$):** This is exactly the same logic as Part 2. Because L is NP-hard, if it can be solved in polynomial time, then every problem in NP can be solved in polynomial time.
- **Direction 2 ($P = NP \rightarrow L \in P$):** This follows from the definition of NP-completeness. By definition, an NP-complete language L is a member of the class NP . If we assume the premise that $P = NP$, then by substitution, L must also be a member of the class P .

In summary, NP-complete problems like **SAT** or **3SAT** serve as the "key" to the P vs. NP question: if we ever find a polynomial-time algorithm for even one of them, we prove that $P = NP$ for the entire class.

The First Example of an NP-complete Problem

The language **TMSAT** is often cited as the first "natural" or "obvious" example of an **NP-complete** problem because it is fundamentally defined as the problem of simulating the behavior of any Turing Machine. While it serves as a critical theoretical bridge in complexity theory, its reliance on the low-level mechanics of Turing Machines makes it less useful for practical applications compared to other NP-complete problems.

Definition and Components of TMSAT

The language is defined as a set of tuples representing a machine, its input, and constraints on the witness and time: $\text{TMSAT} = \{(\alpha, x, 1^n, 1^t) \mid \exists u \in \{0, 1\}^n. M_\alpha \text{ outputs } 1 \text{ on input } (x, u) \text{ within } t \text{ steps}\}$.

The components of this tuple are specifically structured to ensure the problem remains within the bounds of NP:

- α : The binary encoding of a Turing Machine M_α .
- x : The input string for the machine.
- 1^n : A string of n ones (unary notation) representing the length of the certificate or "witness" u .
- 1^t : A string of t ones (unary notation) representing the maximum number of computational steps allowed.

The use of **unary notation** (1^n and 1^t) is a technical necessity to ensure that the input length is large enough so that a simulation can be performed in **polynomial time** relative to the input size.

The NP-Completeness of TMSAT

To be classified as NP-complete, a language must satisfy two conditions: it must be in the class NP , and it must be **NP-hard**.

1. **Membership in NP:** A problem is in NP if there exists a polynomial-time verifier that can check a given certificate. TMSAT is in NP because, given a certificate u , a Universal Turing Machine (UTM) can simulate M_α on (x, u) for t steps to verify if it outputs 1. This simulation is efficient, typically taking $O(t \log t)$ time.
2. **NP-Hardness:** A language is NP-hard if every other language in NP can be reduced to it in polynomial time. TMSAT is NP-hard because any language $L \in NP$ is defined by some polynomial-time machine M and a certificate length $p(|x|)$. One can simply "map" any instance of L into an instance of TMSAT by providing the code for M , the input x , and the appropriate polynomial bounds for n and t .

A Quick Recap on Propositional Logic

- ▶ Formulas of **propositional logic** are either:
 - ▶ Propositional variables, like $X, Y, Z, \dots$;
 - ▶ Built from smaller formulas by way of the connective $\wedge, \vee$ and $\neg$.

Formulas are indicated as $F, G, H, \dots$,

- ▶ Examples: $X \vee \neg X, X \wedge (Y \vee \neg Z)$, etc.
- ▶ Given a formula F and an assignment ρ of elements from $\{0, 1\}$ to the propositional variables in F , one can define the **truth value** for F , indicated as $\llbracket F \rrbracket_\rho$, by induction on F :

$$\begin{aligned} \llbracket X \rrbracket_\rho &= \rho(X) & \llbracket F \vee G \rrbracket_\rho &= \llbracket F \rrbracket_\rho + \llbracket G \rrbracket_\rho \\ \llbracket F \wedge G \rrbracket_\rho &= \llbracket F \rrbracket_\rho \cdot \llbracket G \rrbracket_\rho & \llbracket \neg F \rrbracket_\rho &= 1 - \llbracket F \rrbracket_\rho \end{aligned}$$

- ▶ Examples: $\llbracket X \vee \neg X \rrbracket_\rho = 1$ for every ρ , while the truth value $\llbracket X \wedge (Y \vee \neg Z) \rrbracket_\rho$ equals 1 only for some of the possible ρ .
- ▶ A formula F is **satisfiable** iff there is one ρ such that $\llbracket F \rrbracket_\rho = 1$.
- ▶ A propositional formula F is said to be in **conjunctive normal form** (or a **CNF**) when it is a conjunction of disjunctions of *literals* (a literal being a variable or its negation).
- ▶ Examples: $X \vee \neg X$ and $X \wedge (Y \vee \neg Z)$ are both CNFs, while a formula which is *not* a CNF is $X \vee (Y \wedge \neg Z)$.

- The disjunctions in a CNF are said to be **clauses**, and a **k CNF** is a CNF whose clauses contains at most $k \in \mathbb{N}$ literals. Examples: the two formulas $X \vee \neg X$ and $X \wedge (Y \vee \neg Z)$ are 2CNFs, but not 1CNFs.

What is a CNF (Conjunctive Normal Form)?

A formula is in **Conjunctive Normal Form (CNF)** when it is a **conjunction (AND, $\wedge$) of disjunctions (OR, $\vee$) of literals**,. A literal is simply a propositional variable (e.g., X) or its negation (e.g., $\neg X$),.

Defining Clauses and k CNF

- **Clause:** In a CNF formula, the individual disjunctions (the components separated by the $\wedge$ operator) are called **clauses**. Essentially, each set of literals grouped by $\vee$ operators forms one clause.
- **k CNF:** A CNF formula is classified as a **k CNF** if every single one of its clauses contains **at most k literals**, where k is a natural number. This k value represents the upper limit on the size of any individual clause within the formula.

Analysis of the Specific Examples

Here is why the provided examples are considered 2CNFs but not 1CNFs:

1. $X \vee \neg X$: This formula consists of a **single clause** containing **two literals** (X and $\neg X$). Because the maximum number of literals in its clauses is 2, it is a 2CNF. It cannot be a 1CNF because its only clause exceeds the one-literal limit.
2. $X \wedge (Y \vee \neg Z)$: This formula has **two clauses**: the first is (X), which contains 1 literal; the second is $(Y \vee \neg Z)$, which contains **2 literals**. The "bottleneck" or maximum clause size is 2, making this a 2CNF. It is not a 1CNF because the second clause contains more than one literal.

Theorem (Cook-Levin)

The following two languages are NP-complete:

$$\text{SAT} = \{\ulcorner F \urcorner \mid F \text{ is a satisfiable CNF}\}$$

$$\text{3SAT} = \{\ulcorner F \urcorner \mid F \text{ is a satisfiable 3CNF}\}$$

- Finally, we need to show that $\text{SAT} \leq_p \text{3SAT}$.

Proving a Problem Hard

- ▶ Suppose you are studying the complexity of a language $\mathcal{L}$ and you are convinced that the underlying problem is **hard**. How should you back your claim?
- ▶ By proving that $\mathcal{L} \in \mathbf{P}$?
 - ▶ **Of course not**, this way you rather prove that $\mathcal{L}$ is easy.
- ▶ By proving that $\mathcal{L} \in \mathbf{EXP}$?
 - ▶ **No**, the fact that there is an exponential-time algorithm deciding $\mathcal{L}$ does **not** mean that no polynomial-time algorithm for $\mathcal{L}$ exist.
- ▶ By proving that $\mathcal{L} \in \mathbf{NP}$?
 - ▶ Again, this **does not** mean much.
- ▶ By proving that $\mathcal{L}$ is **NP**-complete?
 - ▶ **Yes**, this way you prove that the problem is not so hard (being in **NP**), but not so easy either (unless $\mathbf{P} = \mathbf{NP}$).

Proving a Problem **NP**-complete

- ▶ If we want to prove $\mathcal{L}$ to be **NP**-complete, we have to prove two statements:
 1. That $\mathcal{L}$ is in **NP**.
 - ▶ This amounts to showing that there are p polynomial and $\mathcal{M}$ polytime TM such that $\mathcal{L}$ can be written as

$$\mathcal{L} = \{x \in \{0, 1\}^* \mid \exists y \in \{0, 1\}^{p(|x|)} . \mathcal{M}(x, y) = 1\}$$
 - ▶ This is typically rather easy.
 2. That any other language $\mathcal{H} \in \mathbf{NP}$ is such that $\mathcal{H} \leq_p \mathcal{L}$.
 - ▶ We can of course prove the statement directly.
 - ▶ More often (e.g. when showing 3SAT **NP**-complete), one rather proves that $\mathcal{J} \leq_p \mathcal{L}$ for a language $\mathcal{J}$ which is already known to be **NP**-complete.
 - ▶ This is correct, simply because $\leq_p$ is transitive:

$$\begin{array}{c}
 \vdots \\
 \mathcal{H} \xrightarrow{\leq_p} \mathcal{J} \xrightarrow{\leq_p} \mathcal{L} \\
 \vdots
 \end{array}$$

So you don't need to prove that there is a reduction between L , and every single *NP* problems, because each problem in *NP* is already proved to be reducible to the other problem in *NP* we just need to reduce L to one problem already in *NP*

NP-complete Problems: Examples

Theorem

The Maximum Independent Set Problem INDSET is NP-complete.

- ▶ There is a polytime reduction from SAT to INDSET.

Theorem

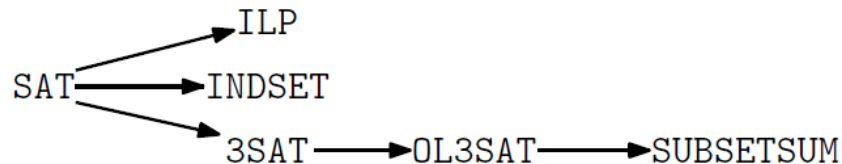
The Subset Sum Problem SUBSETSUM is NP-complete.

- ▶ There are polytime reductions from 3SAT to a variation OL3SAT of it, and from the latter to SUBSETSUM.

Theorem

The Decisional 0/1 Linear Programming Problem ILP is NP-complete

- ▶ There is an easy polytime reduction from SAT to ILP.
- ▶ A fragment, the one we have encountered so far is the following



- ▶ In fact, this graph is **huge**: thousands of different problems are known to be NP-complete

EXERCISES

Exercise 1

Show that matrix multiplication is in FP.

Step for the solution:

- 1) define a pseudo code
- 2) check that the translation from pseudo code to TM don't hide any hidden exponential operation

Matrix multiplication formula

$$f: \left(p, q, r, (A_{ij})_{\substack{1 \leq i \leq p \\ 1 \leq j \leq q}}, (B_{jk})_{\substack{1 \leq j \leq q \\ 1 \leq k \leq r}} \right) \rightarrow \left(\sum_{j=1}^q A_{ij} B_{jh} \right)_{\substack{1 \leq i \leq p \\ 1 \leq k \leq r}}$$



$$f: \left(p, q, r, (A_{ij})_{\substack{1 \leq i \leq p \\ 1 \leq j \leq q}}, (B_{jk})_{\substack{1 \leq j \leq q \\ 1 \leq k \leq r}} \right) \rightarrow \left(\sum_{j=1}^q A_{ij} B_{jh} \right)_{\substack{1 \leq i \leq p \\ 1 \leq h \leq r}}$$



An Algorithm that computes it:

Input(p, q, r, A, B)

Initialise list $C[]$

For i from 1 to p :

Initialise list $C[i][[]]$

For k from 1 to r :

$C[i][k] \leftarrow 0$

For j from 1 to q :

$C[i][h] \leftarrow C[i][h] + A[i][j] \times B[j][h]$

Return C



NOTE:

N is the size of the encoding of the two matrix. The size of the encoding depends on the size of the Matrices and the size of the element inside the matrices

This pseudo code describing a polynomial time TM because:

1. It executes $O(n^3)$ instructions, Because $p, q, r = O(n)$

2. The internal data is C , is it polynomial in n ?

Yes, $|C| \leftarrow p \times r \times (q \times |A| \times |B|)$



$|A| \leftarrow p \times q \times \log(\max A_{ij})$

$\lfloor \log \max A_{ij} \rfloor$

Encoding of the biggest element of A (the encoding is log because it is a binary list)

3. Each instruction is computed in polynomial time

(in particular it is the case of the sum and product $C[i][h] + A[i][j] \times B[j][h]$)

Exercise 2

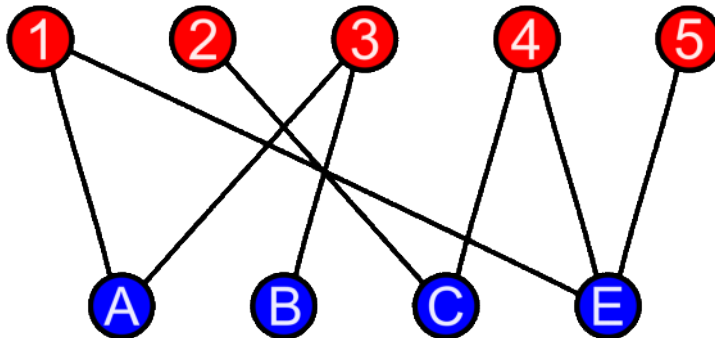
A graph (V, E) is bipartite if $V = V_1 \uplus V_2$ and $\forall (v, w) \in E, v \in V_1$ and $w \in V_2$ or $v \in V_2$ and $w \in V_1$

↑
DISJOINT
UNION

↑
DISJOINT
UNION

or $v \in V_2$ and $w \in V_1$

Example of Bipartite Graph



BASICALLY EACH
NODE IS CONNECTED
ONLY TO A NODE
OF THE OTHER GROUP

Questions:

1. The problem of "determining if a graph is bipartite" is it in P?

This is a particular instance of the graph k -coloring problem. In this case K is fixed at 2-color, It is not passed in input. Because in a Bipartite Graph there are two sides, and the node in a side can be connected only to the node of the other side, we can solve the problem with a BFS when we color with the same color all the node of the same level. If we ever find a node in the same level to have a different color to the one it should have, we fail.

Consider the following algorithm:

Input: G presented as an adjacency matrix $\perp G \perp$

$C \leftarrow [0, \dots, 0]$

$Q \leftarrow []$

For i from 1 to n :

If $c[i] = 0$:

 push i at the end of Q

 while $Q \neq []$:

 Pop j from the top of Q

 For h from 1 to n :

 If $\perp G \perp_{jh} = 1$ or $\perp G \perp = 1$:

 if $C[j] = C[h]$ then

 return false

 if $C[h] = 0$ then

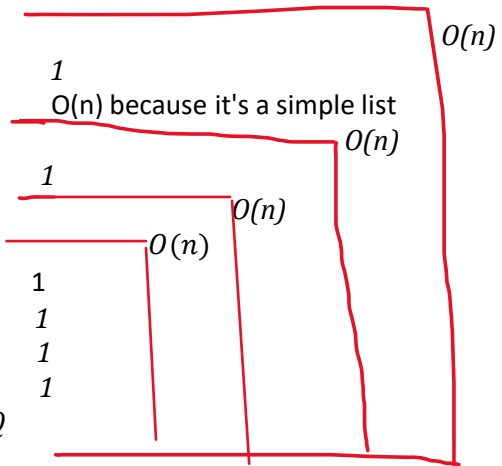
$C[h] \leftarrow 3 - C[j]$

 Push k at the end of Q

Return True

$O(n)$

1



1. Initialization:

- $C \leftarrow [0, \dots, 0]$: Creates an array C of size n , where $C[i] = 0$ means node i is currently uncolored. This takes $O(n)$ steps.
- $Q \leftarrow []$: Initializes an empty queue for the BFS.

2. Outer Loop (For i from 1 to n):

- This loop ensures the algorithm processes every node. This is necessary to handle graphs with **multiple disconnected components**.
 - If $c[i] == 0$: If node i is not yet colored, it marks the start of a new component. (Note: A standard implementation would assign i a starting color like 1 here).
- 3. BFS While Loop** (while $Q \neq []$):
- Pop j from the top of Q : Retrieves the next node to explore.
 - **Neighbor Exploration** (For h from 1 to n): The algorithm scans the j -th row of the adjacency matrix to find all neighbors h of node j .
 - If $G[j][h] == 1$: If an edge exists between j and h :
 - **Conflict Check**: if $C[j] == C[h]$: If a neighbor already has the same color as the current node, the graph is not bipartite. The algorithm returns **False**.
 - **Coloring**: if $C[h] == 0$: If the neighbor is uncolored, it is assigned the opposite color of j using the logic $C[h] \leftarrow 3 - C[j]$ (if j is color 1, h becomes 2; if j is color 2, h becomes 1).
 - **Queueing**: Push h at the end of Q : Adds the newly colored neighbor to the queue to continue the BFS.
- 4. Completion:**
- If all nodes are successfully colored without conflicts, the algorithm returns **True**.

In this algorithm we never insert a node in the Queue if we see that it's already colored

The algorithm is polynomial because:

it executes $O(n^4)$ instructions (note that the size of the input is not n but n^2 , so this algorithm

Has a quadratic complexity $O((n^2)^2)$

Each instruction is polynomial

All the internal data is polynomial: C of Size n

Q of Size n

2. Is it in NP ?

Yes because $P \subseteq NP$

3. Is It NP-complete ?

No, because otherwise we had just proved that $P = NP$

EXERCISE 3

Are these problem in NP ?

(are they NP-hard?)

- 1. Hamiltonian Path**: given a graph, is there a path going through each vertex exactly once

the certificate: a hamiltonian path $\rightarrow$ so this problem is in NP

one can check in polynomial time in the size of the input

Is NP-hard because there is a reduction to the 3-SAT problem.

The professor explain that there is a way to reduce graph to 3-SAT

2. **Eulerian Path:** given a graph, is there a path going through each edge exactly once

The certificate: an eulerian path -> so this problem is in NP
one can check in polynomial time in the size of the input

This problem is actually in P

3. **Travelling Salesman Problem:** given a weighted graph G , and an integer K , is there a hamiltonian cycle in G of total weight $< K$?

The certificate: the path

This problem is NP-hard because it is a complexification of the hamiltonian path, so exist the reduction between those two problems.
(the prof advise us to try to do this prof)

4. **Independent Set:** given a graph (V,E) and an integer k , is there a set $I \subseteq V$ of cardinal K such that $\forall u, v \in I, (u, v) \notin E$

5. Halting problem

This is not a NP problem
But it is an NP-hard Problem (it's harder than every NP problem)

The reduction between Halting problem and NP problem has something to do with cycling throw the solutions

(lesson 20/04/2026, this topic has already been partially-covered in this notes, in the previous pages)

Theorem

1. $<_P$ is a preorder (I.E. it is reflexive and transitive)
2. If L is NP-HARD and in P , then $P=NP$
3. If L is NP-COMPLETE, then $(L \in P)$ iff $P = NP$

Proof

2. Suppose that L is NP-HARD and that L is in P . we can prove, with this HYPOTHESIS, that $NP \subset P$. Take then any language $H \in NP$, and let us try to prove that H is also in P . From the definition of NP-HARDNESS, we can conclude that $H \leq_P L$. We can finally say that H is in P , Because it is at Most as Difficult as L , which is in P .
3. Suppose that L is an NP-COMPLETE problem. Let us then Prove that $L \in P$ iff $P=NP$.

=> Suppose L is in P . We know that L is also NP-COMPLETE, thus NP-HARD. We are then in a situation in which point 2 can be used "OFF-THE-SHELF", concluding that $P=NP$.

<=>

Suppose that $P=NP$ since L is NP-COMPLETE, L is in $NP = P$, the L is also in P itself

Examples of NP-COMPLETE problems

Note: NP hard is a notion that only make sense in decision problems. So it is not a notion for Optimization problem

Knapsack problem

Given a set of objects $\{1, \dots, n\}$, each of them coming with a weight w_i and a price P_i , and given a capacity z for your backpack, determine whether there is a choice of objects $I \subset \{1, \dots, n\}$ such that

$$\sum_{i \in I} w_i \leq z \qquad \sum_{i \in I} P_i \geq r$$

Where r is the target value

Certificate: a binary string of length n representing I

Verifier: on input $\{w_i\}, \{P_i\}, z, r, I$ verify whether these inequalities hold

$$\sum_{i \in I} w_i \leq w \qquad \sum_{i \in I} P_i \geq r$$

Bin-packing

(there is more than one BIN in this problem so it is NOT like the Knapsack)

You have n objects $\{1, \dots, n\}$, Each of them has a size s_i , a bin size $B \in \mathbb{N}$, and another natural number $K \in \mathbb{N}$, you are asked to determine whether there is a partition $x_1, \dots, x_k$ of $\{1, \dots, n\}$ such that

$$\sum_{i \in X_j} s_i \leq B \qquad \forall j \in \{1, \dots, k\}$$

Certificate: it is a tuple of length n whose elements are all in $\{1, \dots, k\}$ the fact that the i -th element of the tuple is s means that $i \in X_s$

Verifier: it checks that the inequality above is satisfied for every j

Cliques

A clique in an undirected graph G is a subset W of V such that each pair of distinct nodes $v, w \in W$ are linked by an edge in the graph. The problem asks you to determine whether a given undirected graph G has a clique of cardinality at least equal to a threshold $k \in \mathbb{N}$

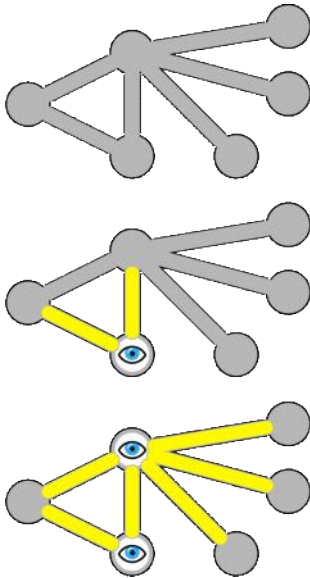
Certificate: it is (an encoding) of W , the subset of V we are looking for

Verifier: Given (G, k) and W , we check whether:

- $|W| \geq k$
- Every pair of distinct nodes $v, w \in W$ are linked by an edge $\{v, w\} \in E$

Vertex cover

Given an undirected graph G and a natural number $k \in \mathbb{N}$, we are interested in checking whether G has a cover of at most k nodes, whereas a cover of a graph is a subset of its nodes such that every edge has at least one of its endpoints in the cover



A vertex cover is a set of Nodes that cover all the edge in the graph

Certificate: a subset of W of V . (the set of nodes that cover the edges)

Verifier: the following algorithm

```
if  $|w| > k$  then:
    Return false
else:
    foreach  $e \in E$  do:
         $\{v, w\} \leftarrow e$ 
        If  $v \notin W$  and  $w \notin W$  then:
            Return false
    Return true
```

The professor's solution focuses on proving the **NP-hardness** of the **NODE COVER** problem (also called Vertex Cover) by creating a bridge between it and the **INDSET** (Independent Set) problem. Here is a detailed breakdown of the logic used in that solution, cross-referenced with the provided source material.

The Complementary Relationship

The core of the proof is a "Lemma" which states that a subset of vertices W is a **vertex cover** if and only if its complement $V \setminus W$ is an **independent set**.

The professor breaks this down using formal logic:

- **Vertex Cover Definition:** For every edge $\{v, w\}$ in the graph, at least one of the endpoints must be in the set W (i.e., $v \in W$ OR $w \in W$).
- **Logical Transformation:** If v is in W , then v is **not** in the complement $V \setminus W$. Therefore, the condition " $v \in W$ or $w \in W$ " is logically equivalent to saying it is **not** the case that both v and w are in the complement set $V \setminus W$.
- **Independent Set Definition:** An independent set is a group of nodes where **no two nodes are connected by an edge**.
- **Conclusion:** If no edge has both endpoints in $V \setminus W$, then $V \setminus W$ must be an independent set.

The Polynomial Reduction ($INDSET \leq_p NODE COVER$)

To prove NODE COVER is NP-hard, we must show that if we could solve NODE COVER, we could also solve INDSET. We do this by defining a transformation function f .

The Logic of the Parameters

The professor's notes show that an instance of INDSET (G, k) —which asks if there is an independent set of size **at least** k —can be transformed into an instance of NODE COVER (G, b) .

The key mathematical relationship is: $|V \setminus W| = |V| - |W|$

According to the source and the conclusion of the professor's notes:

- If a graph has an **Independent Set** W of size $|W| \geq k$...
- ...then the remaining nodes $V \setminus W$ form a **Vertex Cover** of size $|V| - |W| \leq |V| - k$.

Therefore, the reduction function f takes the INDSET instance (G, k) and outputs the NODE COVER instance $(G, |V| - k)$.

Summary of the Proof Steps

1. **Input:** An instance of INDSET consisting of a graph G and an integer k .
2. **Transformation:** We use a machine M to calculate the new threshold $b = |V| - k$. This is a simple subtraction, which is **polynomial time**.
3. **Equivalence:**
 - If G has an independent set of size $\geq k$, it **must** have a node cover of size $\leq |V| - k$.
 - If G has a node cover of size $\leq |V| - k$, it **must** have an independent set of size $\geq k$.
4. **Conclusion:** Since INDSET is NP-complete and we have reduced it to NODE COVER in polynomial time, NODE COVER is **NP-hard**. Combined with the fact that NODE COVER is in **NP** (because a certificate can be verified in polynomial time), the problem is **NP-complete**.

Exam like EXERCISES

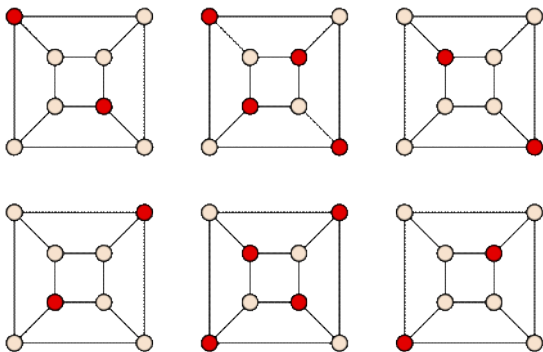
Classify the complexity of the following problems, proving your claim:

SAT where each clause contains 4 literals

- 1) **4SAT** = $\{ F \in 4\text{CNF} \mid F \text{ is satisfiable} \}$
- 2) **3CLIQUES** = $\{ G \mid G \text{ contains a clique of cardinality } 3 \}$
- 3) **Varindset** = $\{ (G, k) \mid G \text{ is a weighted graph, i.e. nodes have a reward, } G \text{ has an independent set of total reward equal to at least } K \}$

The problem **Varindset** is a weighted variation of the **Maximum Independent Set (INDSET)** problem discussed before. To understand it, we must first break down its components: the graph structure, the property of independence, and the weighted reward condition.

Normal maximum independent Set problem: find a set of nodes that are not connected by any edge



- **The Condition (K):** Unlike the standard version of the problem which asks if a set of at least k nodes exists, **Varindset** asks if there is an independent set where the **sum of the rewards** of the chosen nodes is at least K

ex

Imagine a small graph G representing a project selection task:

- **Nodes (Projects):** $\{A, B, C, D\}$
- **Rewards (Profit):** $A = 10, B = 20, C = 15, D = 5$
- **Edges (Conflicts):** ... list of the edges ...
- **Goal (K):** Suppose we want to know if we can achieve a set of node that has a total reward of at least $K = 25$.

- 4) **GenHamiltonian** = $\{ (G,k) \mid G \text{ is a graph whose nodes each have a height } h_v \text{ and such that } G \text{ has a cycle going through each node of height } k \text{ precisely once} \}$

4SAT: solution

4sat is NP-complete indeed:

- 4sat is in NP, because SAT is in NP, and every instance of 4SAT can be checked, together with an assignment of truth values to the underlying, for satisfiability, using precisely the same verifier we know that can be employed for SAT.
- About NP-Hardness, it suffices to notice that every 3CNF can be transformed into an equivalent 4CNF.

Here an example

$$(l_1 \vee l_2 \vee l_3) \rightarrow (l_1 \vee l_2 \vee l_3 \vee X), \\ (l_1 \vee l_2 \vee l_3 \vee \neg X)$$

We add an extra useless literal to every disjunction of the CNF

This transformation preserves the satisfiability. Moreover, it can be computed in polynomial time, we can conclude that 3SAT is $<_p$ than 4SAT

3Cliques: solution

We know that the following problem is NP-Complete:

Cliques = $\{(G,k) \mid G \text{ has a clique of cardinality at least equivalent to } k\}$

Rather than trying to prove that 3Clique is NP-Complete, let us Design a polytime algorithm for it:

Input: G , an undirected graph

Output: 0 or 1 depending on G having (or not) a clique of cardinality 3

Note: $V \setminus \{v\}$, means the set V without v

```
for  $v \in V$  do:
  for  $w \in V \setminus \{v\}$  do:
    for  $z \in V \setminus \{v, w\}$  do
      check whether  $\{v, w, z\}$  is a clique
      if so return 1
return 0
```

The total number of instruction is $O(|G|^3)$

The size of intermediate results is polynomial

The check we do in the body of the inner for loop takes polynomial

Varindset: solution

Varindset is NP-Complete.

It is in NP, because one can define certificates and a verifier exactly as in INDSET: the certificate is the subset w of V we claim to be an independent of reward at least k , while the verifier checks w is indeed independent and computes its total reward.

About NP-Hardness, it is natural to proceed by showing that:

$$\text{indset} \leq_p \text{varindset}$$

We have to find a solution $f: \{0,1\}^* \rightarrow \{0,1\}^*$
Such that $f \in FP$ and $x \in \text{indset} \iff f(x) \in \text{varindset}$

$$(G, k) \rightarrow_f (G', k)$$

Where G' has the same nodes and edges as G , and the rewards in G' are all equal to 1.
 f can be computed in polynomial time, because it produces in output a graph with the same shape as the original, only adding some constant rewards to each node.

Moreover

$$\begin{aligned} (G, k) \in \text{indset} & \iff G \text{ has an i. s. of cardinality at least } k' \\ & \iff G' \text{ as defined above has an i. s. of total reward equal to } k \\ & \iff (G', k) \in \text{varindset} \iff f(G, k) \in \text{varindset}. \end{aligned}$$

Way indset is NP-HARD problem?

To explain why the **Independent Set (INDSET)** problem is **NP-hard**, we must look at how it relates to other hard problems in computational complexity. In complexity theory, a language H is considered **NP-hard** if every language L in the class NP can be reduced to it in polynomial time ($L \leq_p H$).

A standard way to prove this is to take a problem already known to be **NP-complete** (like **3SAT**) and show that it can be reduced to the problem in question. If we can transform any instance of 3SAT into an instance of INDSET such that a solution to one provides a solution to the other, then INDSET must be at least as hard as 3SAT.

The Core Concepts

Before diving into the reduction, it is important to define the two problems involved:

- **3SAT**: The task of determining if a Boolean formula in **3-Conjunctive Normal Form (3CNF)** is satisfiable. A 3CNF formula is a conjunction (AND) of clauses, where each clause is a disjunction (OR) of exactly three literals (a variable or its negation).
- **Independent Set (INDSET)**: In its decision form, this problem asks whether a given undirected graph $G = (V, E)$ contains a subset of vertices $W \subseteq V$ of at least size k , such that no two vertices in W are connected by an edge.

The Reduction: $3SAT \leq_p \text{INDSET}$

We want define a **polynomial-time reduction** that turns a 3CNF formula F into a graph $f(F)$ and a

target size k .

1. Building the Graph $f(F)$

Suppose we have a formula F consisting of n clauses: $c_1 \wedge c_2 \wedge \dots \wedge c_n$.

- **Nodes:** For every literal in every clause, we create a node in the graph. Since there are n clauses and each has 3 literals, the graph will have **$3n$ nodes**.
- **Edges (Intra-clause):** Within each group of 3 nodes representing a single clause c_i , we draw edges between all of them (forming a "triangle" or a 3-clique). This ensures that an independent set can pick **at most one** literal from each clause, because picking two would mean picking two connected nodes.

While it is true that in a 3SAT formula, multiple literals within a single clause *could* be true simultaneously without any problem, the structure of the reduction requires the clique for the following logical reason:

The goal of the reduction is to show that a formula F is satisfiable if and only if the graph $f(F)$ has an independent set of size $k = n$ (where n is the number of clauses)

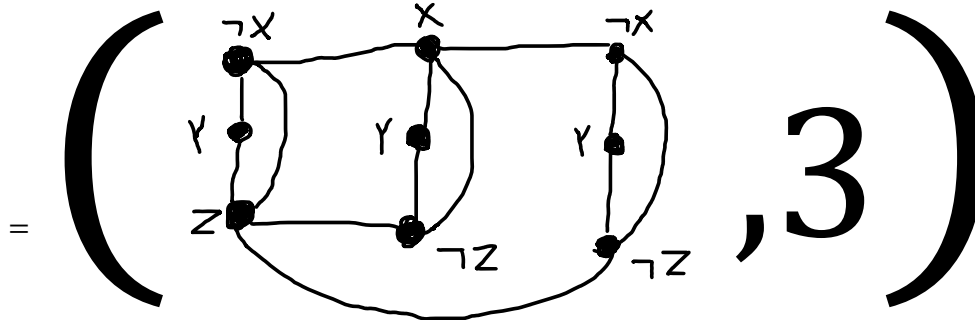
- **Edges (Inconsistency):** We draw an edge between any two nodes in different clauses if they represent **logically incoherent** literals (for example, a node for X and a node for $\neg X$). This ensures that our independent set cannot contain a literal and its negation simultaneously.

2. Setting the Goal k

We set the target size of the independent set to $k = n$ (the number of clauses).

e.g.

$$f((\neg X \vee Y \vee Z) \wedge (X \vee Y \vee \neg Z) \wedge (\neg X \vee Y \vee \neg Z))$$



Proving the Reduction Works

To be a valid reduction, we must show that $F \in 3SAT$ **if and only if** $f(F)$ has an independent set of size n .

- **Forward Direction ($\rightarrow$): If F is satisfiable, then there is an independent set of size n .** If F is satisfiable, there exists a truth assignment ρ such that every clause c_i is true. This means that in every clause, there is at least one literal l_i that evaluates to 1 under ρ . We can pick exactly one such "true" literal from each clause to form a set $W = \{l_1, \dots, l_n\}$.
 - W has size n because we picked one from each of the n clauses.
 - No two nodes in W are connected by an "intra-clause" edge because we only picked one node per clause.
 - No two nodes in W are connected by an "inconsistency" edge because all literals in W are true under the same assignment ρ , so we couldn't have picked both X and $\neg X$. Thus, W is an independent set of size n .
- **Backward Direction ($\leftarrow$): If there is an independent set of size n , then F is satisfiable.** If there is an independent set W of size n :
 - Because the graph is made of n triangles (cliques), the set W must contain exactly one node

from each clause.

- Because W is an independent set, no two selected nodes can be connected. This means the selected literals are logically consistent (we never picked both l and $\neg l$).
- We can therefore create a truth assignment ρ by setting all literals represented in W to true. Because W contains one literal from every clause, this assignment ρ satisfies the entire formula F .

Conclusion

Because this transformation f can be computed in **polynomial time** and correctly preserves the "satisfiability" of the problem, it proves that INDSET is at least as hard as 3SAT. Since 3SAT is known to be NP-complete, this reduction is a witness to the fact that **INDSET is NP-hard**. Furthermore, because we can easily verify if a given set W is an independent set in polynomial time, INDSET is also in the class NP, making it **NP-complete**.

Lesson 30/04/2026

Theorem: 2SAT is in P

Proof

$2SAT = \{ F \mid F \text{ is a 2CNF which is satisfiable} \}$

Remember: implication can be seen as disjunction $A \rightarrow B = \neg A \vee B$

The nice thing about 2cnf is that each clause in it, say $l_1 \vee l_2$ can be seen as the conjunction of two implications:

$$\neg l_1 \rightarrow l_2 = \neg \neg l_1 \vee l_2$$

$$\neg l_2 \rightarrow l_1 = \neg \neg l_2 \vee l_1$$

We can then organize all possible literals possibly occurring in F in a Graph such that:

- Nodes are literals $\neg x$ or x , where x is any propositional variable occurring in F
- Edges are directed and there is one such edge from l_1 to l_2 IF $\neg l_1 \vee l_2$ is a clause in F

Intuition:

if we find such a path between $\neg x$ to x in the Graph, then the Graph is contradictory

why is this graph useful? one can prove that the following two condition are equivalent:

- F is unsatisfiable
- There exist a variable x and a pair of paths in G from x to $\neg x$ and back

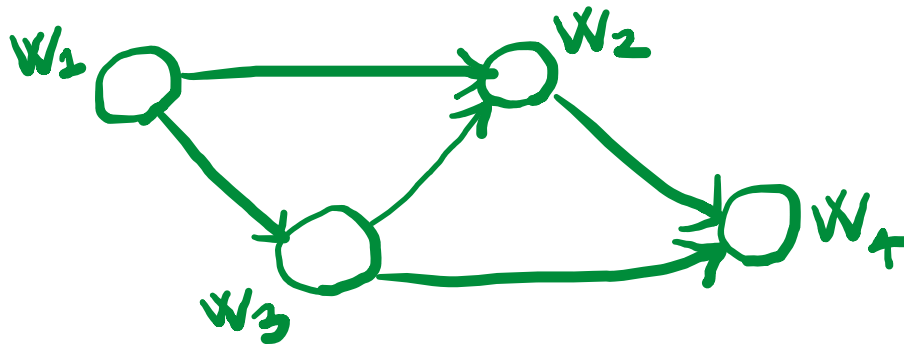
$\Leftrightarrow$ if we know that in G there are paths from x to $\neg x$ and from $\neg x$ to x , then any assignment ρ such that $[F]_\rho = 1$ would be such that if $\rho(x) = 1$ then $\rho(x) = 0$ and if $\rho(x) = 0$ then $\rho(x) = 1$. So there is a contradiction, and such a ρ cannot exist.

$\rightarrow$ this implication holds, but proving it requires building a satisfying assignment ρ from any implication graph without "vicious circles". This can be done by organizing the nodes in the Graph in SCC (strongly-connected components):

Any subset $W \subseteq V$ is a SCC iff for every x, y in W there is a path from x to y . One can then see the Graph as Follows:



then see the Graph as Follows:



Please notice that at a macro-level, this "graph" is acyclic. We can then build the assignment ρ we are looking for reasoning "backward" at the level of SCCs

- DNF are, as we know, Disjunctive normal forms.
- DSAT the problem of checking whether a DNF is satisfiable, is in P, because a DNF $F = D_1 \vee D_2 \dots \vee D_n$ is satisfiable *iff* there is one among the D_i s which is not "contradictory", i.e., it does not contain both a literal and its negation

$$P \subseteq NP \subseteq EXP$$

Is This the End of The Story?

There are other complexity class

Space complexity class

We want to restrict the amount of memory resources used for the computation: we want to limit the size of the intermediate results

Before looking at the definition, take note of the following:

- First, we assume that the TM under consideration has a dedicated read-only input tape, which does not count toward space usage
- Then, we say that the TM works within space $f: N \rightarrow N$ if and only if the number of cells used on the work tapes is at most $f(n)$ for an input $x \in \{0,1\}^*$ of length n .
- Finally, we define **SPACE**(f) as the class of all languages decided by a TM working space proportional to f .
 f is the logarithm function that is the function that for examples helps us to understand how many bits are required to store a specific information. the base of the logarithm function

change by the number of symbols we have in the alphabet (e.g. Binary $\rightarrow$ base 2)

PSPACE class

$$\mathbf{PSPACE} = \bigcup_K \mathbf{DSPACE}(n^K)$$

There are a lot of problems in this class that are known to be in P or NP. For example memory optimizations problems for games (ex game strategies doesn't have a canonical definition of certificate a.k.a a "good move"). There is also a notion of PSPACE-completeness

It also exists:

$$\mathbf{LOGSPACE} = L = \mathbf{DSPACE}(\log n)$$

There is a theorem about the relationship between Logspace and P but I miss it : (

coNP

The Definition of coNP as the Dual of NP

coNP the "dual" to NP. While a language is in **NP** if there **exists** ($\exists$) a certificate that can be verified in polynomial time, a language is in **coNP** if it can be expressed using a **universal quantifier** ($\forall$).

- **NP:** $x \in L$ if there is *some* y such that the verifier accepts (x, y) .
- **coNP:** $x \in L$ if for *all* possible y of polynomial length, the verifier accepts (x, y) .

Concrete Examples: SAT vs. Validity

Here an example of the difference between these classes using propositional logic:

- **SAT (NP-complete):** Asks if there is *at least one* assignment that makes a formula true.
- **Valid CNFs (coNP):** Asks if *every* possible assignment makes the formula true (i.e., the formula is a tautology). This is a typical task for coNP because it requires checking all possible configurations to ensure no "false" result exists.

The Path to the Polynomial Hierarchy (PH)

The text suggests that NP and coNP are just the beginning. By **alternating quantifiers**—switching between "there exists" ($\exists$) and "for all" ($\forall$)—one can define an entire sequence of increasingly complex classes. This structure is known as the **Polynomial Hierarchy (PH)**, which sits between P and PSPACE in the complexity spectrum.

Containment in PSPACE

Notice that **NP** $\subseteq$ **PSPACE** (and by extension, coNP and PH are also within PSPACE). The reasoning is based on the same logic used to prove that **NP** $\subseteq$ **EXP**: a deterministic machine can decide these languages by systematically checking every possible certificate y . While this process might take exponential *time*, it can be done using only a polynomial amount of *space* to store and iterate through the certificates.

Alternative Model of Computation: BPP

It means alternative to deterministic TM.

Randomness as a Computational Resource

The text explains that "efficient" computation is not limited to deterministic algorithms (like those in **P**). By allowing a Turing Machine to make **random choices** (modeled as flipping unbiased coins), we create a **Probabilistic Turing Machine (PTM)**.

Distinction from Nondeterminism

Crucially, this is described as **fundamentally different** from the nondeterministic computation used to define **NP**.

- In **NP**, a machine "accepts" if there exists *at least one* successful path among many.
- In **Probabilistic computation**, we are concerned with the **probability** of success across paths.

The "Bounded Error" Requirement

For a language to be considered "decidable" by a probabilistic machine, it must meet a specific reliability threshold known as **bounded error**:

- **If the answer is YES ($x \in L$):** The machine must accept with a probability of at least **2/3**.
- **If the answer is NO ($x \notin L$):** The machine must reject with a probability of at least **2/3**.
- **Why 2/3?** This specific constant (greater than 1/2) ensures that the "error" is bounded away from pure chance, allowing us to increase our confidence in the answer by running the algorithm multiple times.

The Class BPP

The text defines **BPP** (**B**ounded-error **P**robabilistic **P**olynomial time) as the set of all languages decidable by a PTM in polynomial time with bounded error.

- **Feasibility:** Along with **P**, BPP is often considered a class of "tractable" or "feasible" problems.
- **Relationship to P:** The text notes that $P \subseteq BPP$ (since a deterministic machine is just a probabilistic one that never flips coins). However, it highlights a common observation in complexity theory: most problems found in BPP eventually turn out to be in P, leading many to suspect that randomness might not actually make more problems solvable in polynomial time, even if it makes the algorithms simpler.

Quantum Computing and BQP

Quantum Computation as a Generalization

The most important takeaway is that quantum computation is a **generalization of probabilistic computation**. While the previously discussed probabilistic model (BPP) uses random coin flips to explore different computational paths, a quantum computer uses physical phenomena from quantum mechanics—specifically **superposition**, **entanglement**, and **interference**—to process information in ways that classical bits cannot.

From Bits to Qubits

The transition from classical or probabilistic models to quantum models involves a fundamental shift in the data unit:

- **Classical/Probabilistic:** Uses standard **bits** (0 or 1).
- **Quantum:** Uses **qubits**, which can exist in a superposition of states. A **Quantum Turing Machine** is the theoretical model used to formalize this, where the machine's tapes contain these qubits

instead of classical bits.

The Bounded Error Requirement (The 2/3 Rule)

We define "deciding" a language in the quantum realm using the same **bounded error** logic as the probabilistic class BPP:

- **Acceptance:** If a string x is in the language ($x \in L$), the machine must accept with a probability $\geq 2/3$.
- **Rejection:** If x is not in the language ($x \notin L$), the machine must reject with a probability $\geq 2/3$. This threshold ensures that the result is statistically significant and not just a product of random noise, allowing for the "amplification" of success through repeated trials.

Defining BQP

BQP (Bounded-error Quantum Polynomial time) is the class of all languages that a quantum computer can solve in **polynomial time** with bounded error. In the broader landscape of complexity, this class represents the set of problems that are **efficiently solvable** using the unique advantages of quantum mechanics.

This is the reduction $3DNF \leq_p 3ColoringGraph$

The tutor talk about it very fast I don't think we have to know it

To prove that the **3-Coloring** problem (3COL) is NP-complete, we typically perform a polynomial-time reduction from **3SAT**.

The goal is to take any 3SAT formula ϕ and transform it into a graph G such that ϕ is satisfiable **if and only if** G can be colored using exactly 3 colors. Let's call these three colors **True (T)**, **False (F)**, and **Ground/Base (B)**.

The reduction relies on building specific graph components called **gadgets**. Here is how the reduction works step-by-step:

1. The Base Palette (The Control Gadget)

First, we create a triangle of three connected nodes. We label them **T**, **F**, and **B**.

Because they form a triangle, every vertex must receive a different color. This fixes our palette: one node is forced to be "True", one "False", and one "Base".

2. Variable Gadgets

For every boolean variable x_i present in the 3SAT formula, we create a pair of nodes: one for the literal x_i and one for its negation $\neg x_i$.

- We connect x_i directly to $\neg x_i$ (forcing them to have different colors).
- We connect both x_i and $\neg x_i$ to the **Base (B)** node.

Since they are connected to **B**, neither can be colored with the Base color. Therefore, one must be colored **T** and the other **F**. This perfectly mimics a boolean truth assignment!

3. Clause Gadgets (The Or-Gates)

This is the trickiest part. For each clause containing three literals (e.g., $l_1 \vee l_2 \vee l_3$), we build a specialized gadget made of 6 interconnected nodes that acts like a logical OR gate.

This gadget connects the three literal nodes to the **B** node and the **F** node.

The structural property of this clause gadget is elegant:

- If all three literals l_1, l_2, l_3 are colored **F** (False), the graph **cannot** be 3-colored validly.
- If **at least one** of the literals is colored **T** (True), there exists a valid way to 3-color the rest of the clause gadget.

Summary of the "If and Only If" Proof

- **→ If 3SAT is satisfiable:** We assign colors to the variable nodes based on their true/false values. Because every clause has at least one true literal, the clause gadgets can all be

successfully 3-colored. Thus, the entire graph is 3-colorable.

- ← **If the graph is 3-colorable:** The variable nodes must hold valid, non-conflicting True/False colors because of their connection to the Base node. Furthermore, because the clause gadgets are successfully colored, it guarantees that no clause has all False literals. This gives us a valid truth assignment that satisfies the formula.

Since this graph can be constructed in polynomial time relative to the size of the 3SAT formula, 3-Coloring is proven to be NP-complete.

List of all the NP-hard problem that the professor explain during the lessons

Knowing all those problems is important during the exams for Being able to do the reductions

SAT

(2SAT is in P)

3SAT $\rightarrow$ 4SAT ... ecc

Indset $\rightarrow$ varindset

Integer Linear Programming (ILP)

Hamiltonian Path $\rightarrow$ Travelling salesman

Kcoloring graph (NP complete for $K \geq 3$)

Knapsack

Bin-packing

Cliques (unless the clique dimension is fixed, in that case it is in P)

Vertex cover

Numbers factorization

Subset Sum